

SHIFA-ULLAH

FA24-BCS-146

- Lab Midterm
- Class: BCS-4B
- Subject: Data Structure

For Even registration number students.

Question #1.

Write a Java program to implement a **Singly Linked List (SLL)** and perform the following operations:

1. Create a Singly Linked List by inserting nodes at the end.
2. Add a new node before a given node in the list.
3. Delete any middle node (i.e., a node that is neither the first nor the last).

Your program should include suitable methods for each operation and a main method to demonstrate the functionality of the Doubly Linked List (for example, by creating a list, inserting a node before a specified value, and deleting a middle node).

PROGRAM :-

```

import java.util.Scanner;
public class SinglyLinkedList {
    static class Node
    {
        int data;
        Node next;
        Node(int data)
        {
            this.data = data;
            this.next = null;
        }
    }
    Node head;
    public void insertAtEnd(int data)
    {
        Node newNode = new Node(data);
        if (head == null)
        {
            head = newNode;
            return;
        }
        Node temp = head;
        while (temp.next != null)
        {
            temp = temp.next;
        }
        temp.next = newNode;
    }
    public void insertBefore(int target, int data)
    {
        Node newNode = new Node(data);
        if (head == null)
        {
            System.out.println("List is empty.");
            return;
        }
        if (head.data == target)
        {
            newNode.next = head;
            head = newNode;
            System.out.println(data + " inserted before " + target);
            return;
        }
        Node prev = null;
        Node current = head;
        while (current != null && current.data != target)
        {
            prev = current;
            current = current.next;
        }
        if (current == null)
        {
            System.out.println("Target node not found.");
            return;
        }
        prev.next = newNode;
        newNode.next = current;
    }
}

```

```

        System.out.println(data + " inserted before " + target);
    }
    public void deleteMiddleNode(int value)
    {
        if (head == null || head.next == null)
        {
            System.out.println("No middle node exists.");
            return;
        }

        if (head.data == value)
        {
            System.out.println("First node cannot be deleted.");
            return;
        }
        Node prev = null;
        Node current = head;
        while (current != null && current.data != value)
        {
            prev = current;
            current = current.next;
        }
        if (current == null)
        {
            System.out.println("Node not found.");
            return;
        }
        if (current.next == null)
        {
            System.out.println("Last node cannot be deleted.");
            return;
        }
        prev.next = current.next;
        System.out.println("Middle node " + value + " deleted.");
    }
    public void display()
    {
        if (head == null)
        {
            System.out.println("List is empty.");
            return;
        }
        Node temp = head;
        while (temp != null)
        {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        SinglyLinkedList list = new SinglyLinkedList();
        int choice;
        do
        {

```

```

System.out.println("\n==== Singly Linked List Menu =====");
System.out.println("1. Insert node at end");
System.out.println("2. Insert node before a given node");
System.out.println("3. Delete a middle node");
System.out.println("4. Display list");
System.out.println("5. Exit");
System.out.print("Enter your choice: ");
choice = sc.nextInt();
switch (choice)
{
    case 1:
        System.out.print("Enter value to insert: ");
        int value = sc.nextInt();
        list.insertAtEnd(value);
        System.out.println(value + " inserted.");
        break;
    case 2:
        System.out.print("Enter target node value: ");
        int target = sc.nextInt();
        System.out.print("Enter new value to insert: ");
        int newValue = sc.nextInt();
        list.insertBefore(target, newValue);
        break;
    case 3:
        System.out.print("Enter middle node value to delete: ");
        int deleteValue = sc.nextInt();
        list.deleteMiddleNode(deleteValue);
        break;
    case 4:
        System.out.println("Current List:");
        list.display();
        break;
    case 5:
        System.out.println("Exiting program...");
        break;
    default:
        System.out.println("Invalid choice.");
}
} while (choice != 5);
sc.close();
}
}

```

OUTPUT :-

```
Run SinglyLinkedList
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.1\lib\idea_rt.jar=57490" -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 1
Enter value to insert: 1
1 inserted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 1
Enter value to insert: 2
2 inserted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 1
```

```
Run SinglyLinkedList

5. Exit
Enter your choice: 1
Enter value to insert: 3
3 inserted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 1
Enter value to insert: 4
4 inserted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 1
Enter value to insert: 5
5 inserted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
```

```

Main.java Version Control
SinglyLinkedList.java
Run SinglyLinkedList
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 2
Enter target node value: 5
Enter new value to insert: 6
6 inserted before 5

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 3
Enter middle node value to delete: 3
Middle node 3 deleted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 4
Current List:
1 -> 2 -> 4 -> 6 -> 5 -> null

```

```

Main.java Version Control
SinglyLinkedList.java
Run SinglyLinkedList
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 3
Enter middle node value to delete: 3
Middle node 3 deleted.

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 4
Current List:
1 -> 2 -> 4 -> 6 -> 5 -> null

===== Singly Linked List Menu =====
1. Insert node at end
2. Insert node before a given node
3. Delete a middle node
4. Display list
5. Exit
Enter your choice: 5
Exiting program...

Process finished with exit code 0

```